

6장 Prompt engineering

pretrained model : 사용자의 prompt에 대한 응답으로 text 생성하는 능력 ↑

prompt engineering : 생성된 text의 품질을 향상시키기 위해 prompt 설계하는 방법

6.1 텍스트 생성 모델 사용하기

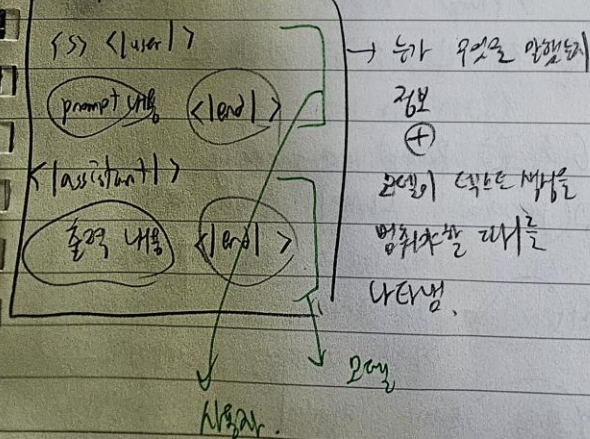
- 특정 모델이 인바디로 성능 ↑
- but open source 모델이 더 유연하고 자유로울 수 있다.
- 작은 크기의 foundation 모델로 시작하는 게 좋다.
- 작은 모델은 가용성을 확보하기에 충분
- ↳ 더 큰 모델로 확장하기 위한 토대

6.1.2 가장 간단한 모델로 코딩하는 방법

= transformer 라이브러리 사용

- transformer pipeline 함수
: 입력 데이터를 처리해서 생성된 텍스트 반환
- 토큰은 <S> : BOS = 문장의 시작
<user> : prompt 시작
<end> : prompt 끝
<assistant> : 출력 시작

template



6.1.3 모델 출력 제어

- 매개변수를 조정하여 출력 제어
temperature

- ① 출력의 다양성 제어
- ② 원하는 prompt, 다른 응답 생성 가능하게 함
- ③ 이를 위해 LLM은 도전을 생성시키거나 각 토큰에 확률값 할당

예) do_sample = False

: 출력이 일정하도록 함

↳ 생성된 X. 확률이 가장 높은 토큰 선택하게 함

do_sample = True

: temperature (top-p 매개변수 사용 가능)

Temperature

: 텍스트의 무작위성 / 창의성 조절

T=0. → 엔지니어 같은 응답 생성
↳ 결정을 지닌 출력 생성

T ↓
T ↑ → 확률이 낮은 단어가 선택될 수 있다.
즉 응답의 다양성 고려 가능

∴ T 덕분에,

모델이 랜덤하게 토큰을 선택하여 출력적으로 동작함
T ↑ → T ↑로 이어짐

top-p

if top-p = 0.4

↳ 유한한 토큰들

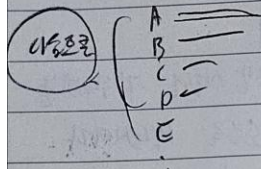
A	0.05
B	0.05
C	0.92
D	0.02

 → 어떤 토큰 : A, B, D

즉, top-p에 지정한 누적합으로 도달할 때까지

top-k

라 하면

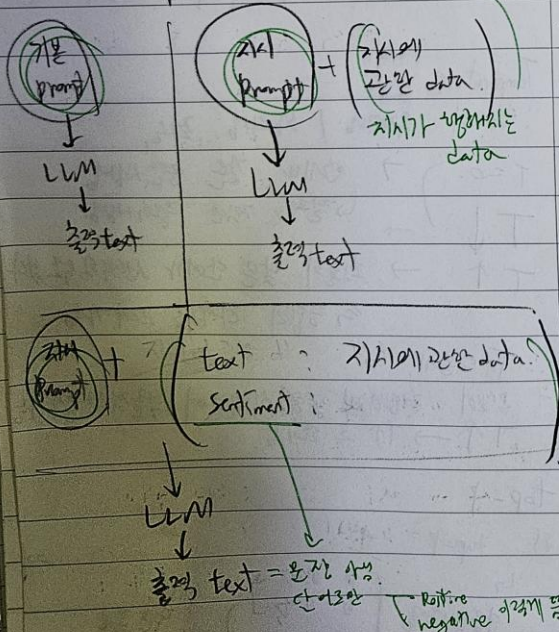


이중 확률이 가장 높은 4개 토큰이 최종 후보 토큰으로 고려됨.

6.2 prompt engineering 소개

주요 목적: 모델로부터 유용한 응답을 이끌어내기
효과적인 prompt를 설계하는 것 이상
모델의 출력 평가 / 안전장치 /
보안 대응도 가능.

6.2.1 Prompt 기본 구성 요소



즉, prompt에 요소를 추가하거나 update 가능.
이런 개념들을 설계하는
것이 프롬프트 엔지니어링.

6.2.2 지시 기반 prompt

Instruction-based prompting

① Bot [지시 지시 관련 data]

② Classification 지시 prompt 문장

Text Sentiment

③ 개체명 인식

개체 정의

개체 기준

- 출력 품질 향상을 위한 기술

① 구체적: 원하는 바를 정확하게 기술

② 포괄적: LLM에 답변을 알아서
생성하라고 모호.

만약 LLM이 답을 안다면, I don't know 라고
응답하게 함.

③ 순서: LLM은 prompt의 시작이나
끝에 정보의 초점을 맞추는 경향이 있다.

→ Prompt 시작이나 끝에 지시사항 편향대라

④ 구체적인 입력의 가장 중요.

6.3 고급 Prompt engineering

① persona: LLM이 수행할 역할 기술
ex) You are expert of math

② 지시: 작업 지시.

가능한 커맨드로 사용

③ 문맥: 문제나 작업에 맥락을 설명하는
추가 정보

④ 형식: LLM이 생성한 text를 출력하는데

사용할 형식

만약에 없으면 LLM이 스스로 해석해서

→ 문맥 생성

⑤ 형식: 생성된 text의 소비대상, 생성된
출력의 사용도 기술

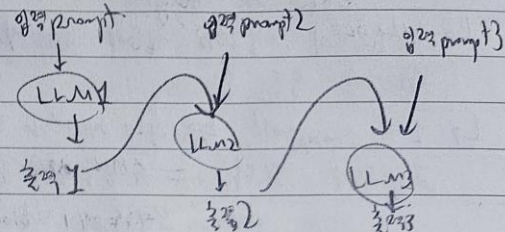
⑥ 어드: LLM이 생성된 text에서 사용할
말

⑦ data: 작업 자체에 관련된 주요 data

6.3.3. prompt chain

- 문제를 여러 prompt로 분해하는 것

- 한 prompt의 출력을 다음 prompt에 입력으로
사용하는 것 → 연속적 chain



- prompt를 직접 test 하자!

→ 커맨드를 자유롭게 추가해서 제거하여
출력에 미치는 영향 판단 가능

→ LLM은 prompt의 초반부 마지막
입력을 중요시함. 즉 prompt 속 내부
커맨드를 간의 순서 - LLM 출력의
품질에 영향 가능

→ 최상의 prompt를 정리하면, 실험이 필수

- 개별 질문에 더 많은 시간 쓰게 함

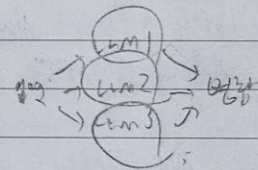
- 각각의 흐름에 매개변수 다르게 지정 가능

ex) 어떤 LLM 출력을 세로로 적기,
비율 및 순서 만들기

어떤 LLM 출력 (행렬)은 세로로 쓰게

ex) 응답 유효성 검사: 이전에 생성한 출력을
재확인하기 LLM에 요청

이별 prompt



이제 작성

6.3.2. 예시제고

In context learning

"LLM에게 우리가 달성하려는 것의 예시를
정확하게 제공 가능"

zero shot prompt: 예시 사용 X

one shot prompt: 1개 예시 사용 O

few shot prompt: 두개 이상 예시 사용 O

*) User(사용자)와 assistant를 구분해줌

→ 만그러면 혼자 하기 나쁘게
보일 수 있음

→ 모델이 제공하는 설명을 활용하기 위한
패턴의 한 조각

6.4 생성모델 활용한 후

후 = 한 지능의 행동

→ 후를 받은 LLM의 생략적 행동
출력? 필요함

출력을 통해 복잡한 동작 보일 수 O

나이를 복원하려고 할 때

즉, prompt engineering으로 인간의 후로 직접
보통 & LLM의 출력 개선 가능

ex) 시스템 1사 vs 시스템 2사

각각의 즉각적인 과정

시스템 1은 더 빠르고
시스템 2는 더 느리고

즉, 여러 생각을 갖고 시뮬레이션 2차원성을
모방하는 셈.

6.4.1 CoT

"Chain of Thought"

→ 생성 모델의 '생각하기' 실용에 대응하게
하는 것이 목적.

→ 입력 prompt에 '추론 관련 예시'가 꼭
포함되어 있다. = 수행할 추론을

보여주는 것이 = thought

→ ~~추론~~ 모델이 두루 과정에 더 많은 계산

→ 정제해답 계산 X, 추론 과정에서 추가된
도론 비정확도 LLM의 안정성/출력안정성

→ 작업에서 모델이 설명하면서 답변
하는 식으로 출력되어 나오므로 함.

Zero shot CoT

→ 입력 예시 X

→ "Let's think step by step"
이런 수행과정을 명시해 놓음.

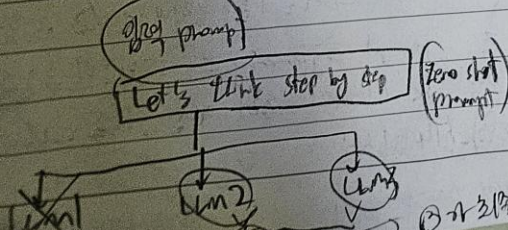
6.4.2 자기 일관성

self-consistency

→ 생성 모델의 출력 향상 시키기 위함.

→ 여러 LLM에 동일한 prompt 보임,
이후 다득점 득점을 가장 높은 투표로
결정하는 최종 답변으로 함.

→ 여러 temperature or top-p 값 사용함



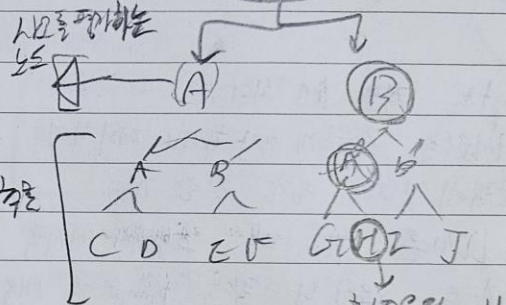
6.4.3 Tree of Thought

Tree of Thought
= 여러 아이디어를 함께 생각

→ 문제를 여러 단계로 나누고
추론에 Good

→ 여러 다른 solution들을 탐색하고 그
다음 최상의 solution 뽑고 다음 단계로 감

ex) 입력 prompt



→ 탐색적인 아이디어 풀음

여러 경로를 고려해야 할 때 큰 도움.

→ but 속도성능이 느려질 수 있음

6.5 출력 검증

= 알려진 A의 출력 생성을 위해, 모델 출력을 검증하고
제어하는 과정. 검증이 필요한 이유

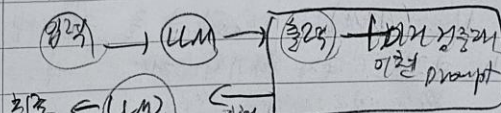
→ 부정확 출력 / 잘못된 출력 / 편향 / 장악성

→ 예시를 보여줌

① 모든 선택과정을 제어하는 방법

② fine tuning으로 데이터에서 학습

ex) LLM의 출력제어를 위해 검증하는 LLM2를 씀



ex) LLM 내부적으로 포맷시키는 '가이드라인' 사용

ex) (Zero prompt) → LLM → 출력